Attackers Learn Twice: Survivorship Bias Hides Within-Attacker Learning in Adaptive Red-Teaming

Anonymous Author(s)
Paper under double-blind review
Submitted to IEEE SaTML

***Abstract*—How quickly do adversaries learn to defeat a defense? The standard answer comes from the aggregate learning curve: attack success rate as a function of attempt index, pooled across attackers. Across two large public corpora of real adaptive human attacks on LLM defenses, LLMail-Inject (461,613 submissions) and Tensor Trust (414,734), that curve falls monotonically: by 60× in Tensor Trust, and in 48 of 48 LLMail defense cells. Taken at face value it says attackers do not learn from failure. We show the curve is upside down. Conditioning on each attacker's own attempt sequence removes between-attacker heterogeneity exactly, and under that conditioning successes arrive late rather than early: the within-attacker hazard *rises* with attempt index (LLMail +0.114, 95% CI [+0.086, +0.143]; Tensor Trust +0.158, [+0.144, +0.171]; three complementary estimators of that position, corroborated by a methodologically distinct split-half test against a within-cell permutation null, $p < 0.001$ on both corpora, and by conditional logistic regression with attacker fixed effects). The aggregate decline is caused by survivorship bias: high-hazard attackers succeed early and leave the risk set. The effect survives holding level fixed and restricting to attackers whose entire sequence falls inside a single six-hour sitting, arguing against community diffusion. Attackers in fact learn twice. First, within a target, by accumulating failures. Conditional logistic regression with attacker fixed effects puts this at a 2.11× increase in the within-attacker *odds* of success per doubling of attempts in Tensor Trust, and 1.04× in LLMail. Second, across successes, by mutation: once an attacker breaks a defense, the next success arrives sooner and is a near-variant of the first (median Jaccard 0.89), forming an *exploit family*. We introduce Family Survival Rate (FSR), the probability that a near-variant of a working payload also passes. FSR measures local robustness under an observed attacker mutation distribution, and attack success rate cannot see it by construction. It spans 11% to 100% across 91 Tensor Trust defenses, though that spread reflects attacker mix as well as defense, and it separates defenses under a within-team paired design on LLMail where attacker identity is held fixed. An LLM-as-judge defense has the lowest FSR on every threshold and target model we test. We also report a clean negative result: the natural hypothesis that boundary determinism drives FSR fails both a cross-corpus replication and a controlled causal test. The practical consequence is that adaptive red-teaming evaluations built on pooled curves systematically understate attacker learning, and evaluations built on ASR alone cannot see exploit families, the mechanism by which real attackers convert one success into many.**

***Index Terms*—adaptive red-teaming, prompt injection, attacker learning curves, survivorship bias, frailty, defense evaluation, large language models**

I. Introduction

Adaptive adversaries are the threat model that matters. A defense that stops a fixed benchmark of attacks but yields to an attacker who iterates is not a defense; it is a delay. The security community has accordingly moved toward evaluating LLM defenses against adaptive attackers who observe outcomes and refine their attempts [1]–[3], and toward public challenges that collect exactly such behaviour at scale [4], [5].

Given such data, the natural question is *how fast attackers learn*, and the natural instrument is the learning curve: the probability that an attempt succeeds as a function of how many attempts have preceded it. Reported statistics in this genre take this form: "80% of successful jailbreaks occur within the first ten attempts", or "attackers need five to seven iterations on average". They are computed by pooling attempts across attackers.

We show that this pooled curve inverts the truth. Section 4 establishes, on both corpora, that the pooled success hazard declines steeply with attempt index while the *within-attacker* hazard rises. The mechanism is classical survivorship bias, well understood in survival analysis under the name *frailty* [6]: when attackers differ in baseline skill, the skilled ones succeed early and exit the risk set, so the population that remains at attempt n is progressively less capable. The aggregate curve tracks the changing composition of the risk set, not the trajectory of any individual in it. A learning curve pooled over heterogeneous attackers is therefore not a learning curve.

Correcting the confound changes the picture in two ways. First, attackers learn from failure after all: within a fixed attacker and a fixed target, the odds of success grow with the attempt index by a factor of 1.04 (LLMail) to 2.11 (Tensor Trust) per doubling. This holds for attackers who eventually succeed; a within-cell design is structurally silent about those who never do (§4.5), so the claim is about the former group only. Second, and more importantly for how defenses should be measured, attackers learn again from *success*. Section 5 shows that once an attacker's first payload gets through, the next success arrives sooner and is a near-variant of the first. Attackers do not rediscover exploits; they mutate the one they have. The working payload seeds an *exploit family*. From that point the attacker's throughput is governed by how much of the family the defense lets through, rather than by the defense's attack success rate.

This motivates the metric of Section 6, **Family Survival Rate**: the probability that a near-variant of a payload that evaded a defense also evades it. FSR is a property of the defense–mutation-distribution pair rather than of the defense alone, since it is defined against whatever variants the attackers

actually produced. It varies by an order of magnitude across defenses, and it is invisible to ASR by construction, because ASR is a marginal statistic that averages over attempts and so discards the dependence among related payloads. On 91 Tensor Trust defenses FSR ranges from 11% at the fifth percentile to 100% at the ninety-fifth; because attackers there chose their own targets, that spread is not a clean ranking of defenses. The comparison that does control attacker identity is on LLMail, where a within-team paired design (the same teams attacking two defenses) finds an LLM-as-judge defense with significantly lower FSR than a classifier-based one on both target models.

Section 7 reports a negative result. The obvious explanation for FSR variation is decision-boundary determinism: a deterministic detector that passes one payload should pass its neighbours, while a stochastic one should not. We tested this two ways and it failed both. It does not replicate: Tensor Trust defenses are near-deterministic (89–96% pass rate on exact resubmission) yet have only moderate FSR. It also fails a controlled causal experiment, in which raising the sampling temperature of a fixed judge moved family survival upward rather than downward. We report this because the hypothesis is natural enough that others will have it, and because the reason it fails is instructive: randomization removes neighbourhood structure, but it also hands the attacker independent re-rolls, and the second effect is at least as large.

Contributions.

1) A demonstration, on 876,347 real adaptive attacks across two independent corpora, that the pooled attacker learning curve is confounded by frailty to the point of inverting sign; a frailty-proof estimator that recovers the within-attacker trend; and robustness of that trend to three stopping-rule treatments, a methodologically distinct split-half estimator with a within-attacker permutation null, level fixation, and calendar-time restriction (§4).
2) Evidence that attackers learn by mutation of successful payloads into exploit families, replicated on both corpora (§5).
3) Family Survival Rate, a metric that captures this mechanism, with a paired-design validation on LLMail showing it separates defenses that ASR cannot (§6).
4) A hypothesis-driven negative result on the determinism hypothesis, including a methodological pitfall (a temperature-invariant scoring rule) that produced an uninformative null on the first attempt (§7).

II. Related Work

Adaptive attacks. The repeated finding that defenses evaluated against static attacks are routinely broken by adaptive ones [1] has driven the field toward optimization-based red-teaming: gradient methods (GCG), search (TAP, AutoDAN), and reinforcement learning against the defended target (PISmith, GPT-Red). These methods receive dense feedback (scores, full model output, tool-call traces) and learn quickly. Our corpora contain *human* attackers receiving sparse feedback (five boolean flags per attempt in LLMail), which is closer to a deployed threat model and, we show, produces a learning signature that aggregate statistics miss.

Human-adaptive versus automated-adaptive. A parallel line of work builds automated adaptive attackers: reinforcement-learning red-teamers trained against the defended target (PISmith, AutoInject, GPT-Red), search-based methods (TAP, AutoDAN), and agentic frameworks that iterate against live systems. These operate in a different information regime from our subjects. An RL attacker receives dense per-step reward, often including the defender's own score, full model output, and tool-call traces. An LLMail participant received five boolean flags, and a Tensor Trust player saw only the target's reply. Our finding that human attackers extract a rising hazard from failure under that sparse signal is therefore a lower bound on what is extractable. The automated results show what is possible with dense feedback; we show that sparse feedback already suffices.

The distinction matters for FSR. An automated attacker's mutation distribution is set by its search operator and is known to the evaluator; a human's is not. FSR is well defined in both settings, but it is *diagnostic* only in the human case, where the mutation distribution is a property of real adversaries rather than of the tool. Measuring FSR against an automated attacker measures the pair (defense, search operator), which is useful for tuning a specific pipeline but does not generalize across attackers. We would expect an automated attacker with an explicit diversity objective to depress FSR relative to a human, and one doing local hill-climbing to inflate it. Testing that is a natural next step, and it needs no new corpus.

Public attack corpora. LLMail-Inject [4] and Tensor Trust [5] are, to our knowledge, the only public corpora recording sequential attempts by identified attackers against identified defenses at scale. Both papers report aggregate attempt-to-success statistics; the LLMail authors note qualitatively that "adaptation was more prominent for Prompt Shield than others". Neither analyses hazard, within-attacker trend, or payload reuse. Twenty-four papers cite LLMail-Inject; none re-analyses its temporal structure.

Evaluation validity. Recent work questions whether benchmark performance of prompt-injection defenses generalizes [7], [8] and whether defense layering compounds as assumed [9]. We share the concern, but target a different object: the *estimator* applied to adaptive-attack data, rather than the benchmark distribution.

Frailty. Unobserved heterogeneity in survival data, and its tendency to make population hazards decline even when individual hazards do not, is textbook [6], [10]. It has been applied to network survivability [11] but not, so far as we can find, to attacker learning curves in security or ML evaluation.

III. Data

LLMail-Inject (SaTML 2025 challenge) recorded 461,613 submissions from 839 participants attempting indirect prompt injection against an LLM email assistant. The design is fully

crossed. Four defenses (Prompt Shield, a trained classifier; TaskTracker, an activation probe; an LLM judge; and Spotlighting, randomized delimiters that are preventive rather than detective) are crossed with two target models (Phi-3-medium, GPT-4o-mini), an all-defenses arm, four difficulty levels and two phases. Each submission records team, timestamp, and five boolean outcome flags; end-to-end success requires retrieval, evasion of the defense, and a correctly-argumented tool call. Feedback to participants was these five flags only. Overall end-to-end success was 0.8% in phase 1 and 0.3% in phase 2.

Tensor Trust (ICLR 2024) is an online attack-and-defense game in which players write natural-language defenses guarding an access code and attack each other's. The v2 raw dump holds 563,349 attacks; dropping self-attacks and records without a defense identifier leaves 414,734 usable attacks by 3,244 attackers against 4,415 defenses over eight months, against three target LLMs (GPT-3.5-turbo, Claude Instant, PaLM). Success is the model emitting "access granted".

The two corpora differ in objective, target models, attacker population, feedback, difficulty (0.3–0.8% vs 7.9% overall success), and time period. Agreement between them is therefore meaningful.

Unit of analysis. Throughout, a *cell* is one attacker against one fixed defense: (target model, defense, team) in LLMail and (attacker, defense id) in Tensor Trust. All within-attacker statistics are computed inside a cell.

IV. The Learning Curve Is Upside Down

A. The pooled curve

Define the pooled success hazard at attempt index *n* as the fraction of attempts at index *n* that are first successes, among cells that have not yet succeeded by *n*. Figure 1(a,b) plots it. In LLMail it declines in every one of 48 (level × model × defense) cells with enough data, with median log–log slope -0.59. In Tensor Trust it falls from 18.1% at the first attempt to 0.3% beyond the 128th, monotonically across eight bins, in all three target-model arms.

A reader who takes this at face value concludes that attempts after the first few are nearly worthless, and that failure carries no exploitable information.

B. Why the pooled curve cannot be trusted

Let attacker *i* have per-attempt success probability $h_i(n)$. The pooled hazard is

$$h_{\text{pop}}(n) \;=\; E[\, h_i(n) \mid i \text{ has not succeeded before } n \,].$$

If attackers are heterogeneous in baseline hazard, meaning some are simply better, then those with high h_i succeed early and leave the conditioning set, and the survivors at large *n* are increasingly the low-h_i ones. h_{pop} declines even if every h_i is constant, and can decline even if every h_i is rising. This is the frailty effect [10], and it means the pooled curve is uninformative about individual learning without an explicit heterogeneity model.

Rather than model heterogeneity, we remove it.

C. A frailty-proof estimator

Condition on each cell's own attempt count *N* and success count *s*. Under the null of no within-attacker time structure, the *s* success positions are a uniform random size-*s* subset of $\{1,\ldots,N\}$, whose mean is exactly $(N+1)/2$ regardless of *s* or the attacker's skill. The statistic

$$T \;=\; \frac{\text{mean(success positions)} \;-\; (N+1)/2}{N},$$

therefore satisfies $\mathrm{E}[T] = 0$ exactly under the null, for every attacker, and its sign gives the direction of the within-attacker trend: negative if successes cluster early (hazard declining), positive if late (hazard rising). Applied to a truncated window of length *W* rather than the full sequence, the same statistic is centred at $(W+1)/2$ and divided by *W*. Between-attacker heterogeneity cannot move it, because it is computed one attacker at a time against that attacker's own null.

Censoring and informative stopping. Write the observed sequence for cell *c* as $Y_c = (y_1,\ldots,y_{\mathrm{N_c}})$ with $y_n \in \{0,1\}$, and let N_c be determined by a stopping rule τ_c that may depend on the history: attackers quit when satisfied, which in practice means shortly after a success. The stopping time is therefore *informative*, in that N_c is not independent of Y_c, and any statistic evaluated on the whole sequence inherits that dependence. Standard right-censoring machinery does not apply, because the censoring is triggered by the event.

Our truncation targets the endpoint. Let $L_c = \max\{n : y_n = 1\}$ be the last success and take the window $W_c = L_c - 1$. Excluding L_c removes the terminal success that the stopping rule places at the observed boundary by construction, which is the mechanical component of the bias, and conditioning further on $s_c = \Sigma_\{\mathrm{n}{\leq}_c\ y_n$ makes the remaining success positions exchangeable under the null. Both the position statistic *T* and the conditional likelihood of §4.5 are computed on this window. This does not by itself make either quantity independent of the stopping process: whether a cell is observed at all, and how long its window is, may still depend on earlier events in that cell, and the truncated window inherits that selection. We return to what truncation does and does not establish below. The cost is that cells with a single success contribute nothing (their window contains no events) and cells that never succeed contribute nothing under any within-cell design (§4.5). We therefore report the untruncated estimator alongside as an explicit bound: it is biased *toward* our conclusion, and we never rely on it.

One complication remains: attackers tend to stop once they succeed, which by construction places the final success at position *N* and biases the statistic upward. We handle this three ways (Table 1). Estimator A ignores it, which is conservative against the hypothesis we are about to reject but biased toward the one we accept. Estimator B drops cells whose last attempt is a success. Estimator T truncates each cell to the attempts strictly before its last success and evaluates the remaining successes in that window. T is primary.

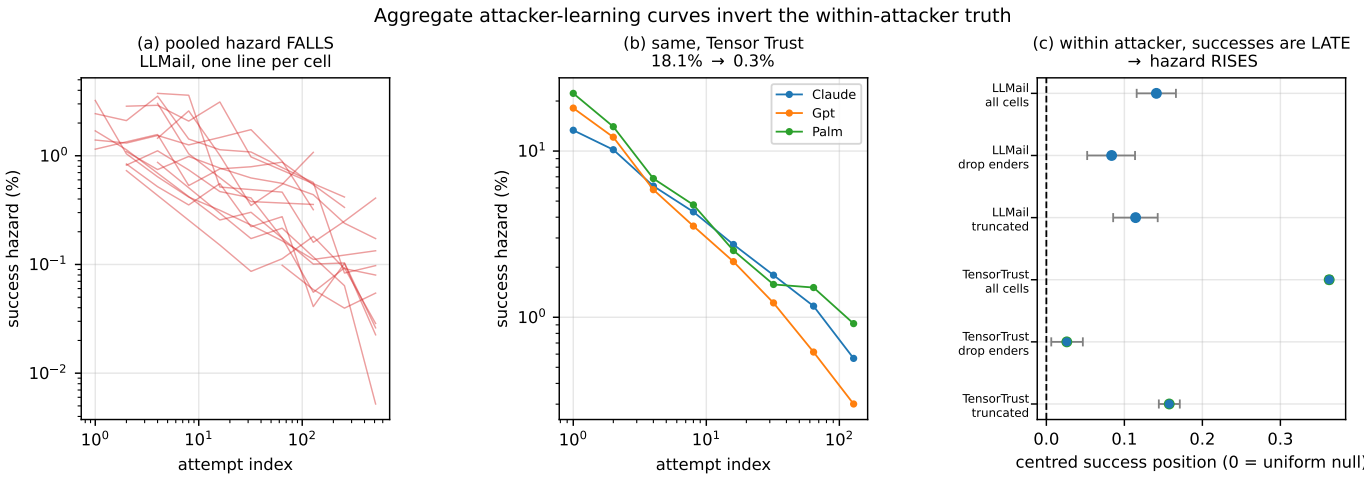


Fig. 1. (a) Pooled success hazard falls in every LLMail cell. (b) Tensor Trust: 18.1% to 0.3%, monotone, all three target models. (c) Conditioning on each attacker's own sequence, successes sit late under every stopping-rule treatment: the within-attacker hazard rises.

Truncation removes the *mechanical* endpoint bias: the terminal success, which the stopping rule places at the boundary by construction, is excluded from the window, so it can no longer inflate the mean position. It does not by itself neutralise every form of informative stopping. Under a history-dependent continuation rule, whether a cell is observed at all, and how long its window is, may still depend on what happened earlier in that cell, and the truncated window inherits that selection. Truncation is sufficient to remove the endpoint artefact exactly; the stronger statement, that the truncated subsequence is independent of the stopping process, would additionally require that continuation depend on the history only through quantities already conditioned on, which we cannot verify from these corpora and do not assume.

We therefore do not rest the result on any single estimator. A, B and T bracket the stopping problem from three directions with different selection properties, but they are three treatments of the same success-position construction applied to overlapping sets of cells, so they are complementary rather than statistically independent and their agreement should not be read as three separate confirmations. The methodologically distinct corroboration comes from the two analyses that share neither that construction nor its conditioning: the split-half statistic of §4.4, which compares success *rates* between halves of a window and is tested against a within-cell permutation null holding N, s and every between-attacker difference fixed, and the conditional-logit model of §4.5, which eliminates the attacker intercept exactly. The confidence in the direction comes from these analyses agreeing, not from a claim that truncation alone solves informative stopping.

All six estimates are positive and exclude zero, and no cell or arm in either corpus produced a negative point estimate. Within an attacker, successes come late, so the hazard rises (Figure 1(c)). The Tensor Trust drop-enders estimator is the weakest of the six, and we do not rest any claim on it.

D. A methodologically distinct estimator and a permutation null

Estimator T is position-based. To rule out an artefact of that construction we use a second estimator sharing none of its form: inside each truncated window, compare the empirical success *rate* in the second half of attempts to the first. We test it against a within-cell label permutation null, which shuffles each attacker's own success labels across their own attempts. This destroys within-attacker time structure while preserving N, s, and every between-attacker difference exactly.

The observed statistic sits 5–15 standard deviations outside a null that preserves frailty exactly. No permutation of 2,000 reached the observed value in either corpus, so the one-sided p is at the resolution floor of $1/(B+1)$ and we report it as an upper bound rather than a point estimate. Frailty alone cannot produce this.

E. Effect size: conditional logistic regression with attacker fixed effects

To put a number on the trend we fit, within each cell,

$$\operatorname{logit} \Pr[\text{success at attempt } n] \;=\; \alpha_c + \beta \log n,$$

and condition on each cell's own success count, which eliminates α_c, and with it attacker skill, exactly. The conditional likelihood sums over all size-s subsets of the window; we evaluate it by the standard clogit dynamic programme in log space, which is exact for every cell including LLMail windows holding 554 successes. Confidence intervals are profile-likelihood. The implementation was checked against brute-force enumeration over all subsets and recovers known β on simulated data with heterogeneous α.

Going from attempt n to attempt $2n$ multiplies the within-cell odds of success by 2.11 for a Tensor Trust attacker and by 1.04 for an LLMail attacker. Both intervals exclude 1. These are odds ratios, the estimand of the fitted conditional-logit

TABLE I
WITHIN-ATTACKER SUCCESS POSITION, EXACTLY CENTRED, WITH 95% BOOTSTRAP CI OVER CELLS. THE STATISTIC IS (MEAN SUCCESS POSITION − $(W+1)/2$) / W OVER A WINDOW OF LENGTH W, WHOSE EXPECTATION IS EXACTLY ZERO UNDER THE UNIFORM NULL.

ESTIMATOR	LLMAIL	TENSOR TRUST
A — all cells	+0.141 [+0.116, +0.166] (377)	+0.363 [+0.359, +0.366] (9,273)
B — drop cells ending in success	+0.084 [+0.052, +0.114] (267)	+0.026 [+0.006, +0.047] (731)
T — truncated window	**+0.114 [+0.086, +0.143] (291)**	**+0.158 [+0.144, +0.171] (1,469)**

TABLE II
SPLIT-HALF RATE DIFFERENCE, SECOND HALF MINUS FIRST, WITH PERMUTATION NULL (2,000 PERMUTATIONS).

CORPUS	WINDOWS	RATE DIFFERENCE	95% CI	NULL MEAN (SD)	ONE-SIDED p
LLMail	286	+0.0265	[+0.0133, +0.0407]	+0.0002 (0.005)	< 0.001
Tensor Trust	1,129	+0.1020	[+0.0873, +0.1171]	+0.0004 (0.007)	< 0.001

TABLE III
CONDITIONAL LOGISTIC REGRESSION WITH ATTACKER FIXED EFFECTS.

CORPUS	WINDOWS	β	ODDS RATIO PER DOUBLING OF ATTEMPT INDEX
LLMail	291	+0.056 [+0.017, +0.097]	1.040× [1.012, 1.069]
Tensor Trust	1,449	+1.077 [+0.999, +1.156]	2.109× [1.998, 2.228]

model; on rare events they approximate a probability ratio, but we report them as odds throughout rather than converting. The LLMail effect is small in magnitude, though its interval excludes 1. The nonparametric results of §4.3 and §4.4 remain the primary evidence for the sign; this section supplies the scale.

Two properties of the estimand deserve statement. First, cells with zero successes (71.8% of LLMail cells and 55.5% of Tensor Trust cells) contribute a likelihood that is constant in β, and are therefore uninformative about a within-cell trend. This is not a filtering choice: *no* fixed-effects design can use them, because a cell with no outcome variation carries no within-cell information. The estimator is silent about attackers who never break through, and our claims are correspondingly about attackers who do. Second, fitting the same model on full sequences rather than truncated windows drives β to the optimizer boundary on Tensor Trust (β = 3.000). That is a direct measure of how severe the stopping-rule bias is, and of why the truncated window is primary. On simulation the estimator carries a mild upward bias in $|\beta|$ at large effect sizes under the $0 < s < W$ conditioning (recovering 0.84 for a true 0.80), so magnitudes should be read as approximate; signs and intervals are unaffected.

F. Ruling out alternatives

Community diffusion. Within an attacker, attempt index and wall-clock time are collinear; a technique spreading publicly mid-challenge would look like individual learning. Restricting to attackers whose entire sequence falls inside a single sitting removes the time for diffusion. At a span of 6 hours or less, Tensor Trust attackers show +0.107 [+0.026, +0.188] (44 windows), the same magnitude as the unrestricted +0.102; at 24 hours or less the estimate is +0.089 [+0.034, +0.143] (92 windows). LLMail at $\leq$ 24 hours: +0.075 [+0.010, +0.148] (43). There is no dose-response with span, which diffusion would produce. Attackers who finish inside a single sitting are of course a self-selected subgroup, so this test bounds the diffusion explanation rather than eliminating every temporal confound.

We note for anyone reproducing this that a *between*-attacker version of the calendar test, comparing high- to low-attempt-index attempts within a calendar bin across attackers, reports the effect reversing. That test is invalid: attackers sitting at high attempt index at any moment are those who have been grinding without success, so it re-measures frailty. It is exactly the error this section corrects.

Level progression. LLMail cells pool difficulty levels 1–4, and teams progress to harder levels over time, which should bias against learning. Holding level fixed, the split-half effect is stronger, not weaker: +0.042 [+0.024, +0.061] over 238 windows.

Clustering. Successes cluster (§5), and one might worry the permutation test conflates clustering with trend. The single-success-window fit of §4.5, in which no clustering is possible, is positive on both corpora.

V. ATTACKERS LEARN FROM SUCCESS, BY MUTATION

The rising hazard of §4 says attackers extract information from failure. They extract far more from success.

The second success comes faster. Among cells with at least two successes, we compare the gap from the first success to the second against the gap to the first. In Tensor Trust the second gap is shorter for 46–62% of attackers, against a per-cell constant-hazard null of 24–29%, with $p < 0.001$ in each of the three target-model arms. In LLMail the same statistic is 64.7% pooled over 286 teams in 15 cells, against that corpus's

own constant-hazard null of approximately 48%; the per-cell values range from 20% to 100%, and two of the fifteen cells fall below 50%. Tested individually against their own nulls, 2 of 8 phase-1 LLMail cells reach significance (both Prompt Shield arms, $p < 0.001$), three are marginal at $p \approx 0.07$, and three are not significant, which we attribute to per-cell samples of 11 to 72 teams. The direction is consistent across cells; the LLMail evidence for this claim is weaker than the Tensor Trust evidence, and the replication rests on the latter.

It is a variant, not a rediscovery. In LLMail phase 2 (the phase with payload text), the second successful payload has median Jaccard similarity 0.89 to the first, but only 14% are exact copies. Attackers take the payload that worked and mutate it. Team 2's own write-up in the LLMail appendix describes the process: an LLM was used "to repeatedly generate new email bodies that matched the working email structure but different in wording", yielding "one successful bypass for every 10–15 email variants".

We find no evidence of transfer across defenses under this measure. For the 29 teams that attacked two defenses, the gap to first success on the second defense is shorter than on the first 48% of the time. With 29 teams this statistic is too imprecise to separate a modest transfer effect from none, so we record it as an absence of evidence rather than evidence of absence. What we can show transfers is the exploit, through the family structure of §6, rather than a general skill.

Family construction. Within a cell we take the first end-to-end success as the family seed and admit every *subsequently submitted* payload within the similarity threshold of that seed; membership is defined against the seed, not transitively, so a family cannot drift arbitrarily far from its origin by chains of small edits. Because families are seeded per cell, one attacker attacking two defenses produces two independent families, and a payload text submitted by two different attackers belongs to both of theirs. For the judge experiment of §7 we de-duplicate at the level of payload text before inference, collapsing 18,315 (family, payload) pairs to 9,392 distinct strings, then re-expand afterwards, so a string shared across families is scored once and counted in each. Exact resubmissions of the seed are tracked separately from near-variants throughout and never counted as family members.

A working payload is thus the seed of an *exploit family*, and after the first success an attacker's throughput is governed by how much of that family the defense lets through.

VI. Family Survival Rate

A. Definition

Let P be a payload that evaded defense D, and let $V(P)$ be the set of subsequently submitted payloads within Jaccard token similarity 0.7 of P. The **family survival rate** of D is

$$\mathrm{FSR}(D) \;=\; \Pr[\, v \text{ evades } D \mid v \in V(P),\; P \text{ evaded } D \,],$$

estimated over all families and reported with a family-level bootstrap.

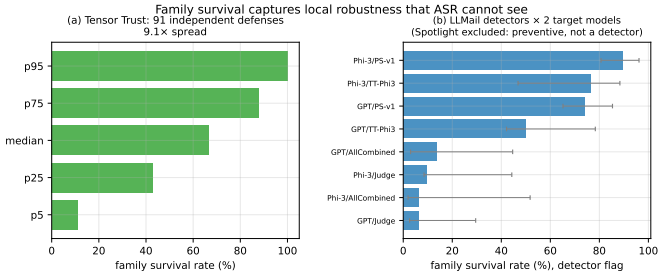


Fig. 2. Family survival rate, which measures local robustness under an observed attacker mutation distribution. (a) 91 Tensor Trust defenses span 11% to 100%. (b) LLMail detectors on both target models; Spotlight is excluded because it is preventive and the detector flag is true for it by construction.

Two admissible seed definitions. "P evaded D" can mean P got past the detector, or that P achieved end-to-end success (retrieved, evaded, and fired the tool with correct arguments). The first is the narrower statement about D; the second is what the attacker was actually trying to do. Both are defensible and they give materially different numbers, so we report the sensitivity rather than choosing silently. The pooled table below uses the detector seed, matching the definition above; the paired analysis of §6.3 uses the end-to-end seed, which is the more conservative of the two for our claim. FSR measures local robustness in the neighbourhood of a defense's own failures, under the mutation distribution the attackers happened to use; it is therefore a property of the defense–mutation-distribution pair, not of the defense in isolation. ASR cannot see it: ASR treats attempts as exchangeable, and FSR is a statement about their dependence.

FSR is a property of a (defense, mutation-distribution) pair. The neighbourhood $V(P)$ is populated by whatever variants the attackers actually submitted, so FSR confounds the defense's local behaviour with how those particular attackers mutate. This matters differently in our two corpora. Tensor Trust is uncrossed, in that each defense faces a different, self-selected set of attackers. Its cross-defense spread (§6.2) therefore mixes defense effects with attacker mix, and we report it as evidence that FSR varies widely in the wild rather than as a ranking of defenses. LLMail is crossed, and the paired design of §6.3 holds the attacker constant on both sides of every comparison; that is where the claim about defenses rests. Any FSR reported against an automated attacker measures the pair (defense, search operator) and should be read accordingly (§2).

B. FSR varies by an order of magnitude

On LLMail, with the detector-level evasion flag and target model held fixed (Figure 2b):

Under the end-to-end seed the same pooled quantities are 36.6% / 50.1% (LLM judge, GPT / Phi-3), 38.1% / 88.5% (Prompt Shield), and 43.8% / 84.4% (TaskTracker). The judge remains lowest on both target models under both definitions, but its margin over Prompt Shield on GPT is large under one seed (6.3% vs 74.2%) and slight under the other (36.6% vs

TABLE IV
FAMILY SURVIVAL RATE ON LLMAIL, DETECTOR-LEVEL EVASION FLAG.

DEFENSE	PHI-3	GPT-4O-MINI
Prompt Shield	89% [81, 96]	74% [65, 85]
TaskTracker	77% [47, 88]	50% [42, 78]
LLM judge	10% [8, 44]	6% [3, 30]

38.1%). We therefore do not claim a magnitude for the pooled comparison; §6.3 is where the defense claim is tested.

On Tensor Trust, across 91 independent defenses with at least ten near-variants (Figure 2a), the 5th percentile is 11.0%, the median 66.7% and the 95th percentile 100.0%, a 9.1× spread. This is not an artefact of four hand-built defenses.

The ten-variant cutoff is a precision floor, and it selects heavily: of 906 defenses with at least one near-variant, it retains 91 (10.0%). The direction of that selection runs against the claim rather than for it: the 815 excluded defenses have a mean FSR of 92.2%, against 62.4% for those retained. The reported distribution is therefore conservative with respect to the claim that family survival is often high. (The excluded estimates are individually near-worthless, being computed on one to nine variants and piling up at 0 and 1; the comparison establishes direction, not magnitude.) The spread itself is not an artefact of the cutoff: it is 9.3× at a cutoff of 20 defenses and 6.1× at 50.

How much of this is the defense, and how much the attacker? Because Tensor Trust is uncrossed we can bound the answer by comparing dispersions. Per-defense FSR ($n \geq 10$) has standard deviation 0.292 across 91 defenses; per-*attacker* FSR, computed identically, has standard deviation 0.269 across 56 attackers. Attacker identity accounts for very nearly as much variation as defense identity. We therefore do not read §6.2 as a ranking of defenses. It establishes that family survival varies widely in deployed conditions, which is the claim that matters for evaluation practice. The defense-level claim rests entirely on the paired design below.

C. It survives a paired design

Teams chose which LLMail defenses to attack, so cross-defense comparisons are confounded by who attacked what. Restricting to teams that attacked *both* members of a pair, and computing each team's FSR on each, gives a paired comparison in which the attacker is their own control (Figure 3). Seven of twelve defense pairs differ significantly. The LLM judge has lower FSR than Prompt Shield on both target models: −32.4 points [−49.3, −16.4] on GPT-4o-mini and −6.7 [−13.1, −2.0] on Phi-3.

This is the one comparison we rely on, so we repeat it under the alternative seed definition. With the detector seed, the same paired contrast gives −53.4 points [−71.5, −32.9] on GPT and −42.9 [−65.6, −20.0] on Phi-3, and 15 of 17 pairs differ (the larger pair count reflects Spotlight becoming comparable under that flag, though its detector-level FSR is 100% by construction and we exclude it from ranking

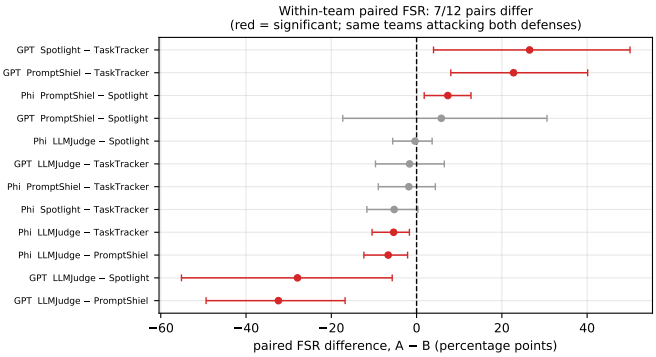


Fig. 3. Within-team paired FSR on LLMail: the same teams attacking both defenses of a pair. Seven of twelve pairs differ; intervals excluding zero are marked.

claims). Judge-below-Prompt-Shield is thus significant in all four combinations of target model and seed definition, with the point estimate ranging from −6.7 to −53.4 points. Sign and significance are robust; magnitude is not, and we claim only the former.

D. What is and is not robust

Token-set Jaccard is order-blind, and a reader may reasonably worry it admits structural rewrites alongside superficial rewords. Recomputing FSR under a character 5-gram measure and an order-sensitive sequence-ratio measure over token sequences, at thresholds 0.7 and 0.85, the LLM judge and the all-defenses arm remain the two lowest under every measure. Absolute values shift by a few points; the ordering claim does not.

The claim that the LLM judge has the lowest family survival holds on both target models, in the paired design, and at every near-variant threshold from Jaccard 0.5 to 0.9. The *full ordering* among the remaining defenses does not: Prompt Shield versus TaskTracker reverses between target models and shifts with the threshold. We therefore claim only the former.

VII. A NEGATIVE RESULT: DETERMINISM DOES NOT EXPLAIN FSR

The natural hypothesis is that FSR is governed by decision-boundary determinism. A deterministic classifier that passes P should pass small perturbations of P; a stochastic judge, sampling its verdict, has no stable neighbourhood to exploit. Two LLMail measurements are consistent with this: on exact resubmission of a payload that passed, Prompt Shield passes it again 99.7% of the time and TaskTracker 96%, while the LLM judge passes it only 51–71% of the time, and the judge has the lowest FSR.

The hypothesis fails two tests.

It does not replicate. Tensor Trust defenses are near-deterministic, passing 89–96% of exact resubmissions across all three target models, yet their pooled FSR is only about 30%. Determinism is therefore at most necessary, and not sufficient.

It fails a causal test, and the failure is directional. We replayed the 242 real LLMail exploit families (16,469 near-variants) through an open-weights judge (Qwen2.5-1.5B-Instruct) at four temperatures, holding the family set, scoring rule and threshold identical across arms. Sampling was verified to be real and graded: decision consistency across draws falls monotonically with temperature.

Every pairwise contrast between arms has overlapping intervals, but pairwise testing is the wrong instrument for an ordered dose-response: it discards the ordering and pays a multiplicity cost for nothing. Regressing FSR on temperature and bootstrapping over families gives a slope of **+5.10 FSR points per unit temperature, 95% CI [+3.15, +6.67], with P(slope $\leq$ 0) $<$ 0.001**. The mechanism predicts a *negative* slope. This is evidence *against* it, not merely absence of evidence for it.

Nor is it an artefact of one judge size. Repeating the two-temperature contrast with Qwen2.5-7B-Instruct gives FSR 10.73% [4.7, 20.2] at temperature 0 and 11.43% [5.3, 21.1] at temperature 1.0, the same direction, with consistency falling from 0.997 to 0.903. Both judges come from one model family and share a single prompt, so the result is established for these judges and this prompt; we do not claim it for guardrails built differently.

A plausible reading is that two effects compete. Randomizing the boundary removes the neighbourhood structure a found exploit sits in, which helps the defender, but it also gives the attacker an independent re-roll on every variant, including ones a deterministic judge would always have caught. In the judge configurations tested here the second effect dominates: each increment of randomization buys the attacker more in re-rolls than it costs in lost structure. We therefore find no support for randomizing a detector's decision boundary as a way to suppress exploit families, and in these configurations it moved family survival in the opposite direction. Whether that holds for guardrails built differently is untested.

A pitfall worth recording. Our first run of this experiment scored the judge by the ratio Pr[YES] / (Pr[YES] + Pr[NO]) from first-token log-probabilities and thresholded at 0.5. That quantity is exactly temperature-invariant, because temperature scaling raises both logits to the power $1/T$, which cannot change which is larger. The "sampled" arm was therefore not sampled (consistency 0.990 against 0.995) and returned an uninformative null. Scoring from the judge's actually emitted verdict is required. We expect this mistake to be common.

VIII. Discussion

For evaluation. Any statistic of the form "attackers succeed within k attempts" or "success rate at attempt n", computed by pooling over attackers, inherits the frailty confound and will understate individual learning, potentially to the point of reversing its sign, as here. The frailty-proof estimators of §4 cost nothing beyond attacker identifiers, which both public corpora already provide. We recommend that adaptive-attack evaluations report within-attacker statistics or not report learning curves at all.

For defense design. ASR answers "what fraction of attempts get through", marginalising over the relationship between them. Against an adaptive attacker who mutates successes, the operative question is "once one gets through, how many of its neighbours do", and that is FSR. Two defenses with the same ASR can differ tenfold in FSR. Because FSR is measured against a particular mutation distribution, it characterises a defense only relative to the attacker population that produced the variants. The LLM judge in LLMail has both the lowest ASR and the lowest FSR, so the two agree there; but the paired analysis shows they disagree among the other defenses, and on Tensor Trust FSR spans nearly the whole unit interval. We suggest FSR be reported alongside ASR whenever adaptive-attack data are available.

For the mechanism. We do not know what governs FSR. Determinism is not merely insufficient: in the configurations of §7 the dose-response runs significantly in the opposite direction, so randomizing a detector's boundary is not a reliable way to suppress exploit families. A descriptive correlation between FSR and the similarity of surviving variants (r = +0.52 over eight defense-cells) hints that FSR may track how well a defense's decision boundary aligns with the attacker's mutation space, with trained surface classifiers aligned to token-level edits and generative decisions less so. Eight points are not evidence, and we leave this open.

IX. Limitations

Both corpora are challenge or game data; participants may behave differently from adversaries with real incentives, and LLMail's five-flag feedback is richer than a silent production deployment provides. Our near-variant threshold is a token-set Jaccard at 0.7; the "judge lowest" claim is robust from 0.5 to 0.9, under order-sensitive and character-n-gram similarity, and under both family-seed definitions, while the finer ordering and all magnitudes are not. FSR values depend materially on whether a family is seeded by detector evasion or by end-to-end success (the judge's pooled FSR on GPT is 6.3% under one and 36.6% under the other), which is why we claim only the sign of the paired contrast. FSR is defined against the mutation distribution the attackers happened to use, so the Tensor Trust spread partly reflects attacker mix; only the LLMail paired comparisons hold the attacker fixed. The ten-near-variant cutoff on Tensor Trust selects defenses that attackers persisted against; we report the retained fraction and the excluded defenses' FSR rather than assuming the selection is benign. Within-cell fixed-effects estimation is structurally silent about attackers who never succeed (71.8% of LLMail cells, 55.5% of Tensor Trust), so the learning claim is about attackers who eventually break through. The LLMail odds ratio is small (1.04$\times$ per doubling) though its interval excludes 1, and the conditional-likelihood estimator shows mild upward bias in $|\beta|$ at large effect sizes on simulation. The causal experiment covers two judges (1.5B and 7B) and four temperatures, but one prompt and one model family; whether the positive slope holds for a differently-constructed guardrail is untested. Its calibration set is 160 unique benign

TABLE V
TEMPERATURE DOSE-RESPONSE. CONSISTENCY IS THE PROBABILITY THAT TWO DRAWS AGREE ON THE SAME PAYLOAD; FSR IS MEASURED OVER THE SAME 242 FAMILIES IN EVERY ARM.

TEMPERATURE	CONSISTENCY	FSR	95% CI
0.0	0.996	8.65%	[4.6, 14.7]
0.3	0.934	8.73%	[4.7, 14.8]
0.7	0.824	11.04%	[6.4, 18.0]
1.0	0.725	13.64%	[9.2, 19.4]

emails. The threshold question dissolves under sampled-label scoring: the judge emits a verdict rather than a score, so values are binary and every threshold in (0,1) induces the same partition; the 0.5-floor and strict FPR-matched analyses return identical numbers. vLLM inference at temperature 0 is not bit-reproducible under shuffled batching (3.2% of scores identical) though decisions agree 99.5%; we report the "deterministic" arm as near-deterministic. Finally, one methodological pitfall is documented in §7 because it is reproducible and easy to repeat: scoring an LLM judge by a normalised two-way log-probability ratio and thresholding it is temperature-invariant, so an experiment varying temperature under that scoring rule cannot detect an effect of temperature and will return an uninformative null.

X. CONCLUSION

The learning curve the field has been reading is a survivorship curve. Corrected for heterogeneity, human attackers against LLM defenses learn from failure within a target, and learn from success within an exploit family by mutating a payload that worked into near-variants of it. The first fact means adaptive-attack evaluations built on pooled statistics understate the threat. The second means that attack success rate, a marginal statistic that averages over attempts and so discards the dependence among related payloads, cannot see the mechanism by which one success becomes many. We offer family survival rate, which conditions on membership in an exploit family and is measured against an observed attacker mutation distribution, as a metric that can.

XI. OPEN SCIENCE

The artifact accompanying this paper contains the analysis code, the preprocessing scripts that build the derived corpora from the public raw dumps, the result files backing every table and figure, the raw per-payload judge outputs behind §7, the figure-generation scripts, a dependency specification, and a verification harness.

Both datasets are public and are not redistributed here. LLMail-Inject is available from the Hugging Face dataset `microsoft/llmail-inject-challenge` under the MIT licence [4]; Tensor Trust is available from the authors' repository [5]. The artifact's README gives the download commands and the preprocessing steps that turn the raw dumps into the derived files the analyses read.

`python verify.py` checks every headline quantity in this paper and reports PASS or FAIL per quantity. `python verify.py --rerun` additionally re-executes the CPU analysis pipeline from the derived corpora before verifying, so a reviewer can confirm that the numbers reported here are the numbers the code produces. Producing the §7 judge decisions in the first place requires a GPU, but the raw per-payload outputs of all six arms are included in the artifact, together with the payload-to-family index they are analysed against. Every §7 quantity reported here, including the per-arm decision consistency, the family survival rates and their intervals, the temperature slope and the second-judge replication, is recomputed from those raw outputs each time `verify.py` runs rather than read back from a stored summary, so §7 can be checked without a GPU.

XII. LLM USAGE CONSIDERATIONS

Editorial use. LLMs were used for editorial purposes in this manuscript, and all outputs were inspected by the author to ensure accuracy and originality.

Extent of assistance. LLM tools provided supporting assistance across an estimated 15–20% of the overall research workflow. This figure is a good-faith estimate of materially AI-assisted effort rather than a precisely measured quantity. The assistance was concentrated in implementation and preparation rather than in the design or interpretation of the study: writing and debugging portions of the analysis and preprocessing code, implementing statistical estimators to specification, searching and screening related literature, drafting and copy-editing assistance on the manuscript, organising the repository, and preparing and testing the reproducibility artifact. The research question, the study design, the choice of estimands, the interpretation of results, and the decisions about what the evidence does and does not support are the author's.

Responsibility and validation. The author is solely responsible for this submission. The author independently reviewed the analysis methodology, validated the scientific claims, verified all reported quantitative results against the reproducible analysis pipeline, checked bibliographic references against primary sources, reviewed the experimental outputs, and takes full responsibility for the accuracy, originality and integrity of the manuscript and results.

Safeguards against AI-introduced error. Because AI-assisted intermediate outputs can be incorrect, reported quantitative results were subjected to scripted verification rather than transcription, estimators were tested against synthetic cases with known ground truth before being applied, and every bibliographic entry was confirmed against a primary source.

Two errors are worth recording because they are instructive rather than routine. One was methodological and is reported in §7: scoring the judge by a normalised two-way log-probability ratio is exactly temperature-invariant, so the first version of that experiment could not have detected the effect it was testing for; it was caught by the manipulation check, and the raw outputs of both the invalid and the corrected runs are included in the artifact. The other was a bibliographic entry whose author list was incorrect in an intermediate draft and was corrected against Crossref and DBLP. The verification harness accompanying this paper exists in part because such checks are necessary.

LLMs as objects of study. The models named in §7 are experimental subjects, not tools of the analysis. The §7 judges are Qwen2.5-1.5B-Instruct and Qwen2.5-7B-Instruct, served locally with vLLM. The corpora analysed throughout were produced by human attackers against LLM-based systems: LLMail-Inject targets Phi-3-medium-128k-instruct and GPT-4o-mini, and Tensor Trust targets GPT-3.5-turbo, Claude Instant 1.2 and PaLM chat-bison.

Why open-weight models were required. The §7 experiment manipulates sampling temperature and measures decision consistency, which requires control over the decoding path, a reproducible greedy mode, repeatable local inference, and access to the verdict token the judge actually emits. Hosted APIs do not guarantee deterministic decoding and do not expose the emitted verdict in the form the measurement needs, so an API judge could not have supported the temperature contrast. The 1.5B model was chosen after the 7B model proved unusable for this measurement at a matched operating point, where it flagged essentially every payload and left no variation in which to observe family survival; the 7B model is retained as the second-judge replication, where a two-temperature contrast is still informative.

Computational footprint. All §7 runs together used approximately three GPU-hours on a single rented consumer GPU (NVIDIA RTX 4090), comprising roughly 300,000 judge inferences over 9,392 distinct payloads at four temperatures with repeated draws. No model was trained or fine-tuned; all use was inference with published open-weights checkpoints. Compute was limited by de-duplicating payloads before inference (18,315 pairs collapse to 9,392 distinct strings), by making runs resumable so no work was repeated after an interruption, and by running a 200-payload timing probe before committing to each full run. Every other analysis in this paper runs on a laptop CPU in minutes. The corpora were collected by prior work, so this study required no new data collection from human participants.

REFERENCES

[1] M. Nasr, N. Carlini, C. Sitawarin, S. V. Schulhoff, J. Hayes, M. Ilie, J. Pluto, S. Song, H. Chaudhari, I. Shumailov, A. Thakurta, K. Y. Xiao, A. Terzis, and F. Tramèr, "The attacker moves second: Stronger adaptive attacks bypass defenses against LLM jailbreaks and prompt injections," *arXiv preprint arXiv:2510.09023*, 2025.

[2] C. Yin, R. Geng, Y. Wang, and J. Jia, "PISmith: Reinforcement learning-based red teaming for prompt injection defenses," *arXiv preprint arXiv:2603.13026*, 2026.

[3] E. Wallace, C. A. Choquette-Choo, N. Kandpal, S. Toyer, D. Hunn, S. Lin, Y. Wen, X. Qi, C. Wolff, Z. Wang, M. Nasr, S. Zhu, C. Guo, J. F. Cerón Uribe, K. Wang, A. Low, K. Xiao, and K. Chen, "GPT-Red: Automated red teaming via self-play at scale," *arXiv preprint arXiv:2607.26115*, 2026.

[4] S. Abdelnabi, A. Fay, A. Salem, E. Zverev, K.-C. Liao, C.-H. Liu, C.-C. Kuo, J. Weigend, D. Manlangit, A. Apostolov, H. Umair, J. Donato, M. Kawakita, A. Mahboob, T. H. Bach, T.-H. Chiang, M. Cho, H. Choi, B. Kim, H. Lee, B. Pannell, C. McCauley, M. Russinovich, A. Paverd, and G. Cherubin, "LLMail-Inject: A dataset from a realistic adaptive prompt injection challenge," *arXiv preprint arXiv:2506.09956*, 2025.

[5] S. Toyer, O. Watkins, E. A. Mendes, J. Svegliato, L. Bailey, T. Wang, I. Ong, K. Elmaaroufi, P. Abbeel, T. Darrell, A. Ritter, and S. Russell, "Tensor trust: Interpretable prompt injection attacks from an online game," in *International Conference on Learning Representations (ICLR)*, 2024.

[6] T. A. Balan and H. Putter, "A tutorial on frailty models," *Statistical Methods in Medical Research*, vol. 29, no. 11, pp. 3424–3454, 2020.

[7] M. Fomin, "When benchmarks lie: Evaluating malicious prompt classifiers under true distribution shift," *arXiv preprint arXiv:2602.14161*, 2026.

[8] S. Li, C. Yu, Z. Ni, H. Li, C. Peris, C. Xiao, and Y. Zhao, "Defenses against prompt attacks learn surface heuristics," *arXiv preprint arXiv:2601.07185*, 2026.

[9] A. Alotaibi, M. S. Jabbar, S. Al-Azani, and M. Ahmed, "Layered LLM defenses as an ensemble: Access tiers, inference cost, and the measured failure correlation between defense layers," *arXiv preprint arXiv:2608.28327*, 2026.

[10] J. W. Vaupel, K. G. Manton, and E. Stallard, "The impact of heterogeneity in individual frailty on the dynamics of mortality," *Demography*, vol. 16, no. 3, pp. 439–454, 1979.

[11] Z. S. Ma, "Frailty modelling for risk analysis in network security and survivability," *International Journal of Information and Computer Security*, vol. 4, no. 3, pp. 276–294, 2011.